下面是SmManager类的函数定义：

```
class SmManager {
   public:
    DbMeta db_;              // 当前打开的数据库的元数据
    std::unordered_map<std::string, std::unique_ptr<RmFileHandle>> fhs_;
    // file name -> record file handle, 当前数据库中每张表的数据文件
    std::unordered_map<std::string, std::unique_ptr<IxIndexHandle>> ihs_;
    // file name -> index file handle, 当前数据库中每个索引的文件
   private:
    DiskManager* disk_manager_;
    BufferPoolManager* buffer_pool_manager_;
    RmManager* rm_manager_;
    IxManager* ix_manager_;

   public:
    SmManager(DiskManager* disk_manager, BufferPoolManager*
    buffer_pool_manager, RmManager* rm_manager, IxManager* ix_manager)
        : disk_manager_(disk_manager),
          buffer_pool_manager_(buffer_pool_manager),
          rm_manager_(rm_manager),
          ix_manager_(ix_manager) {}

    ~SmManager() {}

    BufferPoolManager* get_bpm() { return buffer_pool_manager_; }

    RmManager* get_rm_manager() { return rm_manager_; }

    IxManager* get_ix_manager() { return ix_manager_; }

    bool is_dir(const std::string& db_name);

    void create_db(const std::string& db_name);

    void drop_db(const std::string& db_name);

    void open_db(const std::string& db_name);

    void close_db();
```

```
    void flush_meta();

    void show_tables(Context* context);

    void desc_table(const std::string& tab_name, Context* context);

    void create_table(const std::string& tab_name,
    const std::vector<ColDef>& col_defs, Context* context);

    void drop_table(const std::string& tab_name, Context* context);

    void create_index(const std::string& tab_name,
    const std::vector<std::string>& col_names, Context* context);

    void drop_index(const std::string& tab_name,
    const std::vector<std::string>& col_names, Context* context);

    void drop_index(const std::string& tab_name,
    const std::vector<ColMeta>& col_names, Context* context);
};
```

这里我们不考虑索引，直接对元数据进行分析操作。

整体框架：

`DbMeta db_` 当前的数据库，可以访问数据库中元数据。

`std::unordered_map<std::string, std::unique_ptr<RmFileHandle>> fhs_;`

通过fhs_可以得到当前文件名的文件句柄，对当前文件进行操作。同时也可以知道此时数据库中哪些文件已经被打开了。

实现函数：

我们需要去实现当前类中某些函数的功能：
再实现这些函数之前我们需要了解一下一些系统自带的函数：

传送门

在SmManager类中，你需要实现的函数接口如下：

`open_db(const std::string& db_name)`：系统通过调用该接口打开数据库
在运行系统时，你需要在命令行中输入一个参数`<database name>`，代表当前数据库的名称，系统会创建一个同名文件夹，并在文件夹中存储所有数据库相关的数据文件。系统再次启动时，会打开数据库名称的同名文件夹，并通过读取元数据文件获取当前数据库的相关元数据。 同时，系统还需要打开所有的表文件和索引文件，并将文件句柄加载到系统中。

提示： 可以去参考官方给我们写好了create_db函数，大部分已经写的很清楚了。
可以用`chdir`系统函数进行当前数据库，调用`std::ifstream ofs(DB_META_NAME)`打开数据库存储信息的文件并从中写出数据。

`close_db()`：系统通过调用该接口关闭数据库
在系统服务端关闭时，系统调用该函数，将元数据信息、表数据等落盘，并关闭相关文件。

提示： 先将数据库的信息全部写会磁盘，通过fhs_关闭已经存在的文件，清空所有元数据，再调用chdir()退出当前数据库。

`drop_table(const std::string& db_name)`：删除表
系统调用该函数删除指定名称等表，并删除相关的数据文件，更新元数据信息

提示： 通过fhs_关闭当前表的文件，并从数据库中的所有表中删除当前表，再将信息刷入磁盘中去。